K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

DATABASE MANAGEMENT SYSTEM LAB FILE

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF ENGINEERING AND TECHNOLOGY

APRIL - 2026

SUBMITTED BY:

AADYA DIWAN
ROLLNO - 2401010040

SUBMITTED TO:

DR. MANSI KAJAL
(ASSISTANT PROFESSOR)

BATCH 2024 – 2028

INDEX

S.No	Topic	Date	Signature
1.	**LAB EXPERIMENT 1**		
2.	**LAB EXPERIMENT 2**		
3.	**LAB EXPERIMENT 3**		
4.	**LAB EXPERIMENT 4**		
5.	**LAB EXPERIMENT 5**		
6.	**LAB EXPERIMENT 6**		
7.	**LAB EXPERIMENT 7**		
8.	**LAB EXPERIMENT 8**		
9.	**LAB EXPERIMENT 9**		
10.	**LAB EXPERIMENT 10**		

Lab Sheet: 1

PART A – COLLEGE DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE:

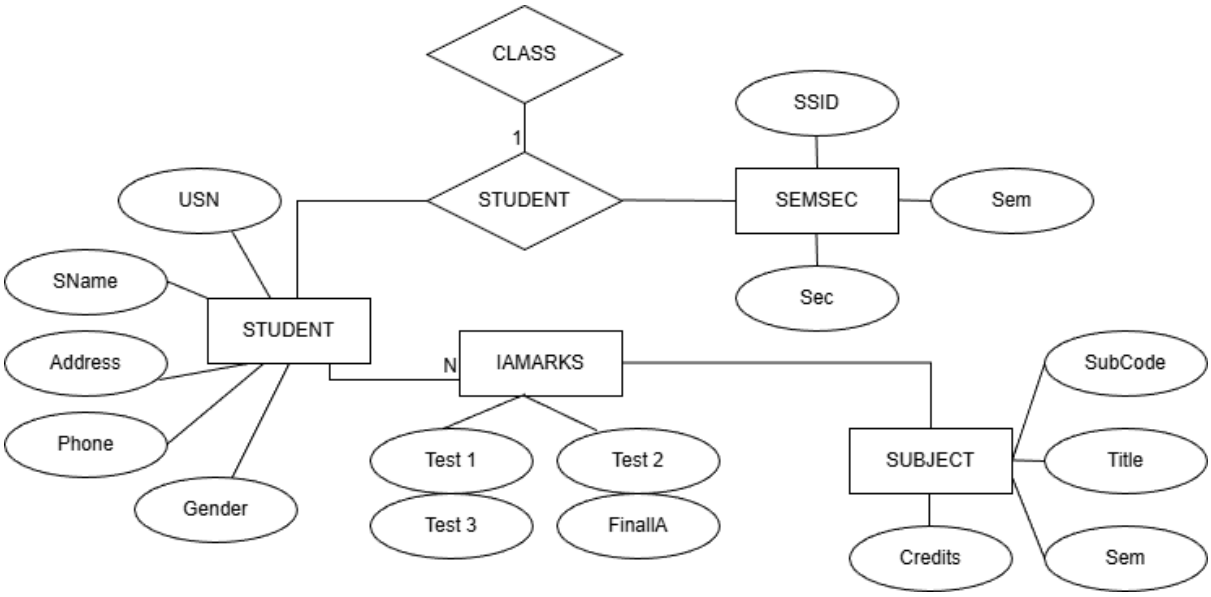
STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)



Explanation of ER Diagram – College Database

1. Introduction

The ER diagram is designed for the College Database system. It represents the structure of the database by identifying entities, their attributes, and the relationships between them. The main aim is to store and manage student details, semester-section information, subject details, and internal assessment marks.

---

2. Entities and Attributes

1. STUDENT

The STUDENT entity represents students of the college.

Attributes:

- **USN** (Primary Key)
- SName
- Address
- Phone
- Gender

USN uniquely identifies each student. No two students can have the same USN.

---

2. SEMSEC (Semester Section)

The SEMSEC entity represents a specific semester and section.

Attributes:

- **SSID** (Primary Key)
- Sem
- Sec

SSID uniquely identifies each semester-section combination.

---

3. SUBJECT

The SUBJECT entity represents subjects offered in different semesters.

Attributes:

- **Subcode** (Primary Key)
- Title
- Sem
- Credits

Subcode uniquely identifies each subject.

---

3. Relationships

1. CLASS Relationship

The CLASS relationship connects STUDENT and SEMSEC.

Meaning:

- Each student belongs to one semester and one section.

- One semester-section can have many students.

Cardinality:

- One SEMSEC → Many STUDENTS (1:M)

This relationship helps to identify which student belongs to which semester and section.

The relation table for CLASS contains:

- USN (Foreign Key)
- SSID (Foreign Key)

---

2. IAMARKS Relationship

The IAMARKS relationship connects STUDENT, SUBJECT, and SEMSEC.

Attributes:

- Test1
- Test2
- Test3
- FinalIA

Meaning:

- A student studies many subjects.
- A subject is studied by many students.
- Marks are recorded for each student in each subject.

Cardinality:

- Many-to-Many (M:N) between STUDENT and SUBJECT.

IAMARKS is considered a relationship with attributes because marks depend on both student and subject.

The relation table IAMARKS includes:

- USN (Foreign Key)
- Subcode (Foreign Key)
- SSID (Foreign Key)
- Test1
- Test2
- Test3
- FinalIA

---

4. How the ER Diagram is Constructed

1. The main entities (STUDENT, SEMSEC, SUBJECT) are identified from the given data.
2. Primary keys are selected to uniquely identify each entity.
3. Relationships are identified:
    - CLASS to connect student and semester-section.
    - IAMARKS to store internal assessment marks.
4. Cardinality is defined:
    - 1:M between SEMSEC and STUDENT.
    - M:N between STUDENT and SUBJECT.
5. Attributes are connected to their respective entities and relationships.

---

5. Conclusion

The ER diagram clearly represents the structure of the college database system. It organizes student details, semester-section details, subject details, and internal assessment marks in a structured manner. The relationships ensure proper linkage between entities and help maintain data consistency.

PART B – COMPANY DATABASE

Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE:

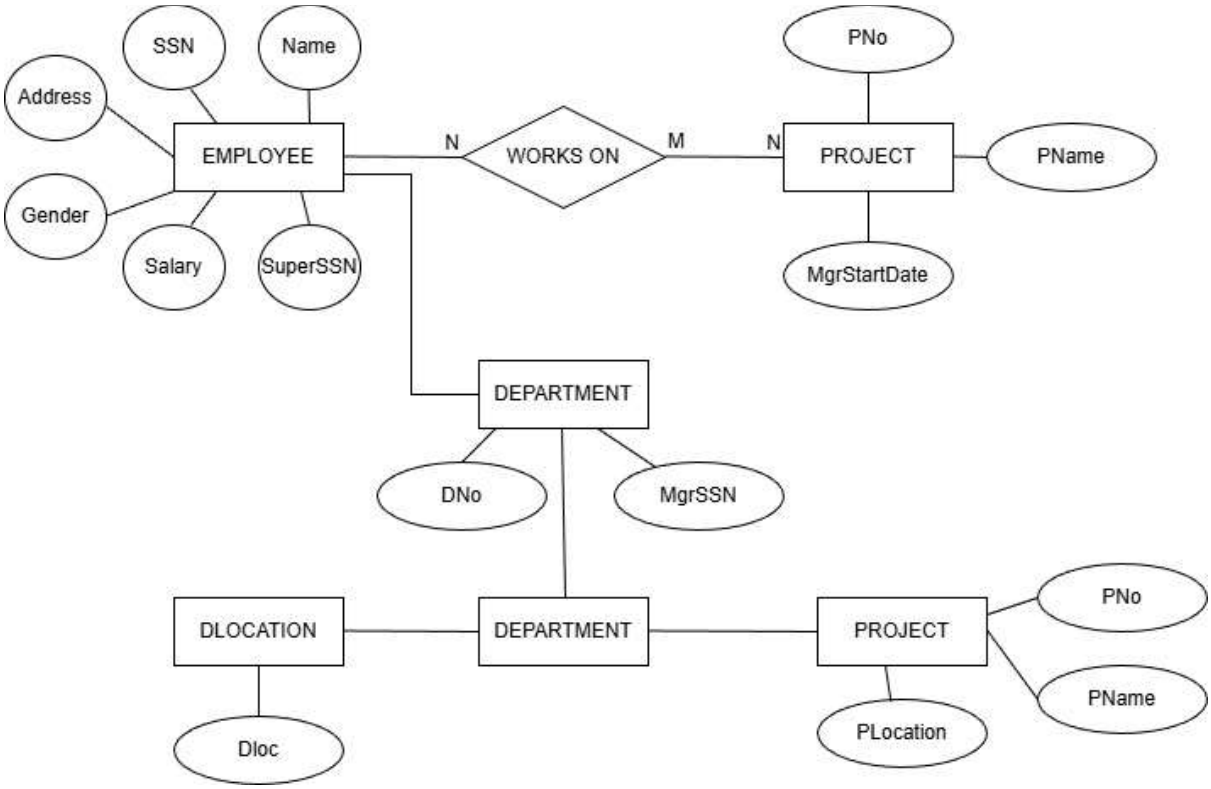
EMPLOYEE (SSN, Name, Address, Sex, Salary, SuperSSN, DNo)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

DLOCATION (DNo,DLoc)

PROJECT (PNo, PName, PLocation, DNo)

WORKS_ON (SSN, PNo, Hours)



Explanation of ER Diagram – Company Database

1. Introduction

The ER diagram represents the structure of a Company Database system. It identifies the main entities such as employees, departments, projects, and locations, along with the relationships between them. The purpose of this ER diagram is to manage employee details, department information, project assignments, and work hours.

---

2. Entities and Attributes

1. EMPLOYEE

The EMPLOYEE entity represents the employees working in the company.

Attributes:

- **SSN** (Primary Key)
- Name
- Address
- Sex
- Salary
- SuperSSN
- DNo

SSN uniquely identifies each employee.
SuperSSN represents the supervisor of the employee (self-referencing relationship).
DNo indicates the department to which the employee belongs.

---

2. DEPARTMENT

The DEPARTMENT entity represents different departments in the company.

Attributes:

- **DNo** (Primary Key)
- DName
- MgrSSN
- MgrStartDate

DNo uniquely identifies each department.
MgrSSN represents the manager of the department.

---

3. PROJECT

The PROJECT entity represents projects handled by the company.

Attributes:

- **PNo** (Primary Key)
- PName
- PLocation

- DNo

PNo uniquely identifies each project.
DNo indicates which department controls the project.

---

4. DLOCATION

The DLOCATION entity represents the locations of departments.

Attributes:

- DNo
- DLoc

A department can have multiple locations.

---

3. Relationships

1. WORKS_ON Relationship

The WORKS_ON relationship connects EMPLOYEE and PROJECT.

Attribute:

- Hours

Meaning:

- An employee can work on multiple projects.
- A project can have multiple employees working on it.

Cardinality:

- Many-to-Many (M:N)

The WORKS_ON table contains:

- SSN (Foreign Key)
- PNo (Foreign Key)
- Hours

---

2. Employee–Department Relationship

- Each employee works in one department.
- One department can have many employees.

Cardinality:

- 1:M (One department → Many employees)

---

3. Department–Project Relationship

- One department controls many projects.
- Each project is controlled by one department.

Cardinality:

- 1:M (Department to Project)

---

4. Department–Location Relationship

- One department can have multiple locations.
- Each location belongs to one department.

Cardinality:

- 1:M (Department to DLOCATION)

---

4. How the ER Diagram is Constructed

1. The main entities (EMPLOYEE, DEPARTMENT, PROJECT, DLOCATION) are identified.
2. Primary keys are assigned to uniquely identify each entity.
3. Relationships are identified:
    - WORKS_ON between EMPLOYEE and PROJECT.
    - Department–Employee relationship.
    - Department–Project relationship.
    - Department–Location relationship.
4. Cardinality is defined:
    - 1:M between Department and Employee.
    - 1:M between Department and Project.
    - 1:M between Department and Location.
    - M:N between Employee and Project.
5. Relationship attributes such as Hours are added to WORKS_ON.

5. Conclusion

The ER diagram clearly represents the structure of the company database. It shows how employees are assigned to departments, how departments manage projects, and how employees work on multiple projects. The relationships ensure proper organization and efficient management of company data.

PART C – CUSTOMER SALE SCHEMA

Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with appropriate integrity constraints

```
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.35 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE CustomerSaleDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE CustomerSaleDB;
Database changed
mysql> CREATE TABLE Customer (
    ->     cust_id INT PRIMARY KEY,
    ->     cust_name VARCHAR(100) NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> CREATE TABLE Item (
    ->     item_id INT PRIMARY KEY,
    ->     item_name VARCHAR(100) NOT NULL,
    ->     price INT NOT NULL CHECK (price > 0)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE Sale (
    ->     bill_no INT PRIMARY KEY,
    ->     bill_date DATE NOT NULL,
    ->     cust_id INT,
    ->     item_id INT,
    ->     qty_sold INT CHECK (qty_sold > 0),
    ->     FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
    ->     FOREIGN KEY (item_id) REFERENCES Item(item_id)
    -> );
Query OK, 0 rows affected (0.09 sec)
```

2. Insert around 10 records in each table

```
mysql> INSERT INTO Customer VALUES
    -> (1,'Aman'),
    -> (2,'Riya'),
    -> (3,'Karan'),
    -> (4,'Sneha'),
    -> (5,'Rahul'),
    -> (6,'Megha'),
    -> (7,'Arjun'),
    -> (8,'Pooja'),
    -> (9,'Vikas'),
    -> (10,'Neha');
Query OK, 10 rows affected (0.03 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Item VALUES
    -> (101,'Pen',20),
    -> (102,'Notebook',80),
    -> (103,'Bag',500),
    -> (104,'Shoes',1200),
    -> (105,'Bottle',250),
    -> (106,'Watch',1500),
    -> (107,'Mobile Cover',300),
    -> (108,'Headphones',2000),
    -> (109,'Keyboard',1800),
    -> (110,'Mouse',400);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Sale VALUES
    -> (1,CURDATE(),1,103,2),
    -> (2,CURDATE(),5,104,1),
    -> (3,CURDATE(),2,101,5),
    -> (4,'2026-02-15',3,105,2),
    -> (5,'2026-02-16',5,108,1),
    -> (6,CURDATE(),6,110,3),
    -> (7,'2026-02-14',4,102,4),
    -> (8,CURDATE(),5,106,1),
    -> (9,'2026-02-12',7,109,1),
    -> (10,CURDATE(),8,107,2);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

3. List all the bills for the current date with customer names and item numbers

```
mysql> SELECT s.bill_no, c.cust_name, s.item_id
    -> FROM Sale s
    -> JOIN Customer c ON s.cust_id = c.cust_id
    -> WHERE s.bill_date = CURDATE();
+---------+-----------+---------+
| bill_no | cust_name | item_id |
+---------+-----------+---------+
|       1 | Aman      |     103 |
|       2 | Rahul     |     104 |
|       3 | Riya      |     101 |
|       6 | Megha     |     110 |
|       8 | Rahul     |     106 |
|      10 | Pooja     |     107 |
+---------+-----------+---------+
6 rows in set (0.02 sec)
```

4. List the total Bill details with quantity sold, price of item and final amount

```
mysql> SELECT s.bill_no,
    ->        s.qty_sold,
    ->        i.price,
    ->        (s.qty_sold * i.price) AS total_amount
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id;
+---------+----------+-------+--------------+
| bill_no | qty_sold | price | total_amount |
+---------+----------+-------+--------------+
|       1 |        2 |   500 |         1000 |
|       2 |        1 |  1200 |         1200 |
|       3 |        5 |    20 |          100 |
|       4 |        2 |   250 |          500 |
|       5 |        1 |  2000 |         2000 |
|       6 |        3 |   400 |         1200 |
|       7 |        4 |    80 |          320 |
|       8 |        1 |  1500 |         1500 |
|       9 |        1 |  1800 |         1800 |
|      10 |        2 |   300 |          600 |
+---------+----------+-------+--------------+
10 rows in set (0.01 sec)
```

5. List the details of customers who have bought a product with price > 200

```
mysql> SELECT DISTINCT c.cust_id, c.cust_name
    -> FROM Customer c
    -> JOIN Sale s ON c.cust_id = s.cust_id
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE i.price > 200;
+---------+-----------+
| cust_id | cust_name |
+---------+-----------+
|       1 | Aman      |
|       5 | Rahul     |
|       3 | Karan     |
|       8 | Pooja     |
|       7 | Arjun     |
|       6 | Megha     |
+---------+-----------+
6 rows in set (0.01 sec)
```

6. Give count of how many products have been bought by each customer

```
mysql> SELECT c.cust_name,
    ->        COUNT(s.item_id) AS total_products_bought
    -> FROM Customer c
    -> LEFT JOIN Sale s ON c.cust_id = s.cust_id
    -> GROUP BY c.cust_name;
+-----------+----------------------+
| cust_name | total_products_bought |
+-----------+----------------------+
| Aman      |                    1 |
| Riya      |                    1 |
| Karan     |                    1 |
| Sneha     |                    1 |
| Rahul     |                    3 |
| Megha     |                    1 |
| Arjun     |                    1 |
| Pooja     |                    1 |
| Vikas     |                    0 |
| Neha      |                    0 |
+-----------+----------------------+
10 rows in set (0.01 sec)
```

7. Give list of products bought by customer having cust_id = 5

```
mysql> SELECT i.item_name
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE s.cust_id = 5;
+------------+
| item_name  |
+------------+
| Shoes      |
| Headphones |
| Watch      |
+------------+
3 rows in set (0.01 sec)
```

8. List the item details which are sold as of today

```
mysql> SELECT i.item_id, i.item_name, i.price
    -> FROM Item i
    -> JOIN Sale s ON i.item_id = s.item_id
    -> WHERE s.bill_date = CURDATE();
+---------+--------------+-------+
| item_id | item_name    | price |
+---------+--------------+-------+
|     103 | Bag          |   500 |
|     104 | Shoes        |  1200 |
|     101 | Pen          |    20 |
|     110 | Mouse        |   400 |
|     106 | Watch        |  1500 |
|     107 | Mobile Cover |   300 |
+---------+--------------+-------+
6 rows in set (0.00 sec)
```

9. Create a view listing bill_no, bill_date, cust_id, item_id, price, qty_sold, amount

```
mysql> CREATE VIEW Bill_Details AS
    -> SELECT s.bill_no,
    ->        s.bill_date,
    ->        s.cust_id,
    ->        s.item_id,
    ->        i.price,
    ->        s.qty_sold,
    ->        (s.qty_sold * i.price) AS amount
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Bill_Details;
+---------+------------+---------+---------+-------+----------+--------+
| bill_no | bill_date  | cust_id | item_id | price | qty_sold | amount |
+---------+------------+---------+---------+-------+----------+--------+
|       1 | 2026-02-18 |       1 |     103 |   500 |        2 |   1000 |
|       2 | 2026-02-18 |       5 |     104 |  1200 |        1 |   1200 |
|       3 | 2026-02-18 |       2 |     101 |    20 |        5 |    100 |
|       4 | 2026-02-15 |       3 |     105 |   250 |        2 |    500 |
|       5 | 2026-02-16 |       5 |     108 |  2000 |        1 |   2000 |
|       6 | 2026-02-18 |       6 |     110 |   400 |        3 |   1200 |
|       7 | 2026-02-14 |       4 |     102 |    80 |        4 |    320 |
|       8 | 2026-02-18 |       5 |     106 |  1500 |        1 |   1500 |
|       9 | 2026-02-12 |       7 |     109 |  1800 |        1 |   1800 |
|      10 | 2026-02-18 |       8 |     107 |   300 |        2 |    600 |
+---------+------------+---------+---------+-------+----------+--------+
10 rows in set (0.00 sec)
```

10. Create a view for daily sales date-wise for last one week

```
mysql> CREATE VIEW Weekly_Sales AS
    -> SELECT s.bill_date,
    ->        SUM(s.qty_sold * i.price) AS total_sales
    -> FROM Sale s
    -> JOIN Item i ON s.item_id = i.item_id
    -> WHERE s.bill_date >= CURDATE() - INTERVAL 7 DAY
    -> GROUP BY s.bill_date;
Query OK, 0 rows affected (0.02 sec)

mysql> SELECT * FROM Weekly_Sales;
+------------+-------------+
| bill_date  | total_sales |
+------------+-------------+
| 2026-02-18 |        5600 |
| 2026-02-15 |         500 |
| 2026-02-16 |        2000 |
| 2026-02-14 |         320 |
| 2026-02-12 |        1800 |
+------------+-------------+
5 rows in set (0.01 sec)
```

Lab Sheet: 2

Table Structures:

CLASS

D (NUMBER) and Name (VARCHAR).

The initial records in the table are (1, 'Bravo'), (2, 'Alex'), and (4, 'Cheng').

EMPLOYEES

Employee_Id (NUMBER), First_Name (VARCHAR), Last_Name (VARCHAR), Email (VARCHAR), Phone_Number (VARCHAR), Hire_Date (DATE), Job_Id (VARCHAR), Salary (NUMBER), Commission_Pct (NUMBER), Manager_Id (NUMBER), and Department_Id (NUMBER).

DDL Questions (Data Definition Language):

1. How do you create a table named CLASS with appropriate data types?

```
MySQL 8.0 Command Line Cli

Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.35 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE SchoolDB;
Query OK, 1 row affected (0.01 sec)

mysql> USE SchoolDB;
Database changed
mysql> CREATE TABLE CLASS (
    ->     ID INT,
    ->     Name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO CLASS VALUES (1,'Bravo');
Query OK, 1 row affected (0.03 sec)

mysql> INSERT INTO CLASS VALUES (2,'Alex');
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO CLASS VALUES (4,'Cheng');
Query OK, 1 row affected (0.01 sec)

mysql>
```

2. How do you create a table named EMPLOYEES with the specified attributes?

```
mysql> CREATE TABLE EMPLOYEES (
    ->     Employee_Id INT,
    ->     First_Name VARCHAR(50),
    ->     Last_Name VARCHAR(50),
    ->     Email VARCHAR(100),
    ->     Phone_Number VARCHAR(20),
    ->     Hire_Date DATE,
    ->     Job_Id VARCHAR(50),
    ->     Salary INT,
    ->     Commission_Pct DECIMAL(5,2),
    ->     Manager_Id INT,
    ->     Department_Id INT
    -> );
Query OK, 0 rows affected (0.04 sec)
```

3. How do you create a table named Client_master with the specified attributes?

```
mysql> CREATE TABLE Client_master (
    ->     ClientNo VARCHAR(10),
    ->     Name VARCHAR(50),
    ->     Address VARCHAR(100),
    ->     City VARCHAR(50),
    ->     State VARCHAR(50),
    ->     Pincode INT,
    ->     Bal_due INT
    -> );
Query OK, 0 rows affected (0.04 sec)
```

4. What SQL statement would you use to rename the table Client_master to Client12?

```
mysql> RENAME TABLE Client_master TO Client12;
Query OK, 0 rows affected (0.04 sec)
```

5. How do you display the bal_due column heading as “BALANCE”?

```
mysql> SELECT Bal_due AS BALANCE FROM Client_master;
+---------+
| BALANCE |
+---------+
|    6000 |
|    4000 |
|    8000 |
|    3000 |
|    9000 |
+---------+
5 rows in set (0.00 sec)
```

DML Questions (Data Manipulation Language):

Part A: Transactions and Savepoints (CLASS Table)

Give the output of the following SQL script:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO CLASS VALUES (5,'Rahul');
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> UPDATE CLASS SET Name='Abhijeet' WHERE ID=5;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO CLASS VALUES (6,'Chris');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO CLASS VALUES (7,'Bravo');
Query OK, 1 row affected (0.00 sec)

mysql> SAVEPOINT C;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM CLASS;
+------+----------+
| ID   | Name     |
+------+----------+
|    1 | Bravo    |
|    2 | Alex     |
|    4 | Cheng    |
|    5 | Abhijeet |
|    6 | Chris    |
|    7 | Bravo    |
+------+----------+
6 rows in set (0.00 sec)

mysql>
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> SELECT * FROM CLASS;
+------+----------+
| ID   | Name     |
+------+----------+
|    1 | Bravo    |
|    2 | Alex     |
|    4 | Cheng    |
|    5 | Abhijeet |
|    6 | Chris    |
+------+----------+
5 rows in set (0.00 sec)

mysql>
mysql> ROLLBACK TO A;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Part B: Retrieving Records (EMPLOYEES Table)

1. Find out the employee id, names, and salaries of all the employees.

```
mysql> SELECT Employee_Id, First_Name, Last_Name, Salary
    -> FROM EMPLOYEES;
+-------------+------------+-----------+--------+
| Employee_Id | First_Name | Last_Name | Salary |
+-------------+------------+-----------+--------+
|         101 | John       | Austin    |   5000 |
|         102 | Neha       | Sharma    |   4500 |
|         103 | Amit       | Verma     |   6000 |
|         104 | Riya       | Austin    |   4800 |
|         105 | Karan      | Mehta     |   7000 |
+-------------+------------+-----------+--------+
5 rows in set (0.00 sec)
```

2. List out the employees who work under manager 100.

```
mysql> SELECT *
    -> FROM EMPLOYEES
    -> WHERE Manager_Id = 100;
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
| Employee_Id | First_Name | Last_Name | Email           | Phone_Number | Hire_Date  | Job_Id  | Salary | Commission_Pct | Manager_Id | Department_Id |
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
|         101 | John       | Austin    | john@gmail.com  | 9876543210   | 2020-01-10 | IT_PROG |   5000 |           0.10 |        100 |            60 |
|         103 | Amit       | Verma     | amit@gmail.com  | 9876543212   | 2018-07-20 | SA_REP  |   6000 |           0.15 |        100 |            80 |
|         105 | Karan      | Mehta     | karan@gmail.com | 9876543214   | 2017-09-18 | MK_REP  |   7000 |           0.12 |        100 |            90 |
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
3 rows in set (0.01 sec)
```

3. Find the names of the employees who have a salary greater than or equal to 4800.

```
mysql> SELECT First_Name, Last_Name
    -> FROM EMPLOYEES
    -> WHERE Salary >= 4800;
+------------+-----------+
| First_Name | Last_Name |
+------------+-----------+
| John       | Austin    |
| Amit       | Verma     |
| Riya       | Austin    |
| Karan      | Mehta     |
+------------+-----------+
4 rows in set (0.01 sec)
```

4. List out the employees whose last name is 'AUSTIN'.

```
mysql> SELECT *
    -> FROM EMPLOYEES
    -> WHERE Last_Name = 'AUSTIN';
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
| Employee_Id | First_Name | Last_Name | Email           | Phone_Number | Hire_Date  | Job_Id  | Salary | Commission_Pct | Manager_Id | Department_Id |
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
|         101 | John       | Austin    | john@gmail.com  | 9876543210   | 2020-01-10 | IT_PROG |   5000 |           0.10 |        100 |            60 |
|         104 | Riya       | Austin    | riya@gmail.com  | 9876543213   | 2021-11-05 | IT_PROG |   4800 |           0.08 |        102 |            60 |
+-------------+------------+-----------+-----------------+--------------+------------+---------+--------+----------------+------------+---------------+
2 rows in set (0.01 sec)
```

5. Find the names of the employees who work in departments 60, 70, and 80.

```
mysql> SELECT First_Name, Last_Name
    -> FROM EMPLOYEES
    -> WHERE Department_Id IN (60,70,80);
+------------+-----------+
| First_Name | Last_Name |
+------------+-----------+
| John       | Austin    |
| Neha       | Sharma    |
| Amit       | Verma     |
| Riya       | Austin    |
+------------+-----------+
4 rows in set (0.01 sec)
```

6. Display the unique Manager_Id.

```
mysql> SELECT DISTINCT Manager_Id
    -> FROM EMPLOYEES;
+------------+
| Manager_Id |
+------------+
|        100 |
|        101 |
|        102 |
+------------+
3 rows in set (0.02 sec)
```

PART C – Updating Records (Client_master Table)

1. Insert five records into the Client_master table.

```
mysql> INSERT INTO Client_master VALUES
    -> ('C123','Ravi','MG Road','Delhi','Delhi',110001,6000),
    -> ('C124','Anita','Park Street','Kolkata','WB',700001,4000),
    -> ('C125','Rahul','Brigade Road','Bangalore','KA',560001,8000),
    -> ('C126','Neha','Marine Drive','Mumbai','MH',400001,3000),
    -> ('C127','Amit','Sector 15','Noida','UP',201301,9000);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

2. Find the names of clients whose bal_due > 5000.

```
mysql> SELECT Name
    -> FROM Client_master
    -> WHERE Bal_due > 5000;
+-------+
| Name  |
+-------+
| Ravi  |
| Rahul |
| Amit  |
| Ravi  |
| Rahul |
| Amit  |
+-------+
6 rows in set (0.00 sec)
```

3. Change the bal_due of ClientNo "C123" to Rs. 5100.

```
mysql> UPDATE Client_master
    -> SET Bal_due = 5100
    -> WHERE ClientNo = 'C123';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

Lab Sheet: 3

Objectives

- **To understand the use of COMMIT and ROLLBACK commands in transaction control.**
- **To use ORDER BY and GROUP BY clauses in SQL queries.**
- **To understand DCL commands (GRANT, REVOKE) and the use of SAVEPOINT.**
- **To understand and implement JOIN operations in SQL.**

DDL Questions (Data Definition Language)

1. Create TEACHER table

```
mysql> create database lab3;
Query OK, 1 row affected (0.02 sec)

mysql> use lab3;
Database changed
mysql> CREATE TABLE TEACHER (
    ->     Name VARCHAR(50),
    ->     DeptNo INT,
    ->     Date_of_Joining DATE,
    ->     DeptName VARCHAR(50),
    ->     Location VARCHAR(50),
    ->     Salary DECIMAL(10,2)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

2. Create SALES table

```
mysql> CREATE TABLE SALES (
    ->     Sales_No INT PRIMARY KEY,
    ->     Salesname VARCHAR(50),
    ->     Branch VARCHAR(50),
    ->     Salesamount DECIMAL(10,2),
    ->     DOB DATE
    -> );
Query OK, 0 rows affected (0.04 sec)
```

3. Create DEPARTMENTS table

```
mysql> CREATE TABLE DEPARTMENTS (
    ->     dept_no INT PRIMARY KEY,
    ->     dept_name VARCHAR(50),
    ->     dept_location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)
```

4. Create EMPLOYEES table

```
mysql> CREATE TABLE EMPLOYEES (
    ->     emp_id INT PRIMARY KEY,
    ->     emp_name VARCHAR(50),
    ->     emp_salary DECIMAL(10,2),
    ->     dept_no INT,
    ->     FOREIGN KEY (dept_no) REFERENCES DEPARTMENTS(dept_no)
    -> );
Query OK, 0 rows affected (0.07 sec)
```

DML (Data Manipulation Language)

From TEACHER Table

1. Insert five records

```
mysql> INSERT INTO TEACHER VALUES
    -> ('Anita', 1, '2015-06-12', 'Mathematics', 'Delhi', 50000),
    -> ('Ravi', 2, '2017-08-20', 'Commerce', 'Mumbai', 45000),
    -> ('Sneha', 1, '2018-03-15', 'Mathematics', 'Delhi', 55000),
    -> ('Amit', 3, '2016-11-01', 'Science', 'Chennai', 48000),
    -> ('Priya', 2, '2019-01-25', 'Commerce', 'Mumbai', 47000);
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

2. 25% increment to Mathematics department

```
mysql> UPDATE TEACHER
    -> SET Salary = Salary + (Salary * 0.25)
    -> WHERE DeptName = 'Mathematics';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

3. 15% increment to Commerce department

```
mysql> UPDATE TEACHER
    -> SET Salary = Salary + (Salary * 0.15)
    -> WHERE DeptName = 'Commerce';
Query OK, 2 rows affected (0.01 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

From SALES Table

1. Insert five records

```
mysql> INSERT INTO SALES VALUES
    -> (101, 'Rahul', 'North', 20000, '1995-12-21'),
    -> (102, 'Meena', 'South', 25000, '1994-05-10'),
    -> (103, 'Arjun', 'North', 18000, '1993-12-09'),
    -> (104, 'Kiran', 'East', 30000, '1996-07-14'),
    -> (105, 'Neha', 'South', 22000, '1992-12-30');
Query OK, 5 rows affected (0.02 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

2. Total sales amount in each branch

```
mysql> SELECT Branch, SUM(Salesamount) AS Total_Sales
    -> FROM SALES
    -> GROUP BY Branch;
+--------+-------------+
| Branch | Total_Sales |
+--------+-------------+
| North  |    38000.00 |
| South  |    47000.00 |
| East   |    30000.00 |
+--------+-------------+
3 rows in set (0.02 sec)
```

3. Average sales amount in each branch

```
mysql> SELECT Branch, AVG(Salesamount) AS Average_Sales
    -> FROM SALES
    -> GROUP BY Branch;
+--------+---------------+
| Branch | Average_Sales |
+--------+---------------+
| North  |  19000.000000 |
| South  |  23500.000000 |
| East   |  30000.000000 |
+--------+---------------+
3 rows in set (0.01 sec)
```

4. Salesname & DOB of salesmen born in December (21-Dec-09 format)

```
mysql> SELECT Salesname,
    ->        DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
    -> FROM SALES
    -> WHERE MONTH(DOB) = 12;
+-----------+-----------+
| Salesname | DOB       |
+-----------+-----------+
| Rahul     | 21-Dec-95 |
| Arjun     | 09-Dec-93 |
| Neha      | 30-Dec-92 |
+-----------+-----------+
3 rows in set (0.01 sec)
```

5. Salesname & DOB in alphabetical order of month

```
mysql> SELECT Salesname, DOB
    -> FROM SALES
    -> ORDER BY MONTHNAME(DOB);
+-----------+------------+
| Salesname | DOB        |
+-----------+------------+
| Rahul     | 1995-12-21 |
| Arjun     | 1993-12-09 |
| Neha      | 1992-12-30 |
| Kiran     | 1996-07-14 |
| Meena     | 1994-05-10 |
+-----------+------------+
5 rows in set (0.02 sec)
```

JOIN Operations

1. Employee and Department details (Inner Join)

```
mysql> SELECT *
    -> FROM EMPLOYEES E
    -> INNER JOIN DEPARTMENTS D
    -> ON E.dept_no = D.dept_no;
+--------+----------+------------+---------+---------+-----------+---------------+
| emp_id | emp_name | emp_salary | dept_no | dept_no | dept_name | dept_location |
+--------+----------+------------+---------+---------+-----------+---------------+
|    101 | Aman     |   50000.00 |       1 |       1 | HR        | Delhi         |
|    102 | Riya     |   60000.00 |       2 |       2 | Finance   | Mumbai        |
|    103 | Kabir    |   55000.00 |       3 |       3 | IT        | Bangalore     |
+--------+----------+------------+---------+---------+-----------+---------------+
3 rows in set (0.00 sec)
```

2. LEFT OUTER JOIN

```
mysql> SELECT E.emp_name, D.dept_name
    -> FROM EMPLOYEES E
    -> LEFT JOIN DEPARTMENTS D
    -> ON E.dept_no = D.dept_no;
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Aman     | HR        |
| Riya     | Finance   |
| Kabir    | IT        |
+----------+-----------+
3 rows in set (0.01 sec)
```

3. RIGHT OUTER JOIN

```
mysql> SELECT E.emp_name, D.dept_name
    -> FROM EMPLOYEES E
    -> RIGHT JOIN DEPARTMENTS D
    -> ON E.dept_no = D.dept_no;
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Aman     | HR        |
| Riya     | Finance   |
| Kabir    | IT        |
+----------+-----------+
3 rows in set (0.00 sec)
```

4. Employees earning more than average salary

```
mysql> SELECT *
    -> FROM EMPLOYEES
    -> WHERE emp_salary > (SELECT AVG(emp_salary) FROM EMPLOYEES);
+--------+----------+------------+---------+
| emp_id | emp_name | emp_salary | dept_no |
+--------+----------+------------+---------+
|    102 | Riya     |   60000.00 |       2 |
+--------+----------+------------+---------+
1 row in set (0.02 sec)
```

TCL (Transaction Control Language)

1. ROLLBACK on TEACHER table

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

2. COMMIT on TEACHER table

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

3. SAVEPOINT implementation

```
mysql> SAVEPOINT sp1;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> ROLLBACK TO sp1;
Query OK, 0 rows affected (0.00 sec)
```

DCL (Data Control Language)

1. Grant all privileges

```
mysql> CREATE USER 'dept_user'@'localhost' IDENTIFIED BY '1234';
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> GRANT ALL PRIVILEGES
    -> ON lab3.EMPLOYEES
    -> TO 'dept_user'@'localhost';
Query OK, 0 rows affected (0.02 sec)
```

2. Grant selected privileges

```
mysql> GRANT SELECT, INSERT
    -> ON lab3.EMPLOYEES
    -> TO 'dept_user'@'localhost';
Query OK, 0 rows affected (0.02 sec)
```

3. Revoke all privileges

```
mysql> REVOKE ALL PRIVILEGES
    -> ON lab3.EMPLOYEES
    -> FROM 'dept_user'@'localhost';
Query OK, 0 rows affected (0.03 sec)
```

4. Revoke selected privileges

```
mysql> REVOKE SELECT, INSERT
    -> ON lab3.EMPLOYEES
    -> FROM 'dept_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

Lab Sheet: 4.1

Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

```
mysql> CREATE DATABASE school_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE school_db;
Database changed
mysql>
mysql> CREATE TABLE class (
    ->     id INT,
    ->     name VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO class VALUES
    -> (1, 'Bravo'),
    -> (2, 'Alex'),
    -> (4, 'Cheng');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Give the output of the following SQL script:

- **INSERT INTO class VALUES (5,'Rahul');**

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> INSERT INTO class VALUES (5,'Rahul');
Query OK, 1 row affected (0.00 sec)
```

- **COMMIT;**

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

- **UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;**

```
mysql> UPDATE class SET name = 'Abhijeet' WHERE id='5';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

- **SAVEPOINT A;**

```
mysql> SAVEPOINT A;
Query OK, 0 rows affected (0.00 sec)
```

- **INSERT INTO class VALUES (6, ‘Chris’);**

```
mysql> INSERT INTO class VALUES (6, 'Chris');
Query OK, 1 row affected (0.00 sec)
```

- **SAVEPOINT B;**

```
mysql> SAVEPOINT B;
Query OK, 0 rows affected (0.00 sec)
```

- **INSERT INTO class VALUES (7, ‘Bravo’);**

```
mysql> INSERT INTO class VALUES (7, 'Bravo');
Query OK, 1 row affected (0.01 sec)
```

- **SAVEPOINT C**

```
mysql> SAVEPOINT C;
Query OK, 0 rows affected (0.00 sec)
```

- **SELECT * FROM class;**

```
mysql> SELECT * FROM class;
+------+----------+
| id   | name     |
+------+----------+
|    1 | Bravo    |
|    2 | Alex     |
|    4 | Cheng    |
|    5 | Abhijeet |
|    6 | Chris    |
|    7 | Bravo    |
+------+----------+
6 rows in set (0.00 sec)
```

- **ROLLBACK TO B;**

```
mysql> ROLLBACK TO B;
Query OK, 0 rows affected (0.00 sec)
```

- **SELECT * FROM class;**

```
mysql> SELECT * FROM class;
+------+----------+
| id   | name     |
+------+----------+
|    1 | Bravo    |
|    2 | Alex     |
|    4 | Cheng    |
|    5 | Abhijeet |
|    6 | Chris    |
|    7 | Bravo    |
|    5 | Abhijeet |
|    6 | Chris    |
+------+----------+
8 rows in set (0.00 sec)
```

- **ROLLBACK TO A;**

```
mysql> ROLLBACK TO A;
Query OK, 0 rows affected (0.00 sec)
```

Lab Sheet: (4.2)

Consider the following table: Table: ACCOUNT_TXN

Acc_ID	Holder_Name	Balance
101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

```
INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);

SAVEPOINT S1;

UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;

UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;

SAVEPOINT S2;

DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;

INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);

SAVEPOINT S3;

UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID
= 102;

SELECT * FROM ACCOUNT_TXN;

ROLLBACK TO S2;

SELECT * FROM ACCOUNT_TXN;

INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);

SAVEPOINT S4;

UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN
(101,104);

SELECT * FROM ACCOUNT_TXN;
```

ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

COMMIT;

Student Tasks

Write the output of all 4 SELECT statements in tabular format

Identify which rows are permanently saved after COMMIT

Mention which operations were undone by:

ROLLBACK TO S2

ROLLBACK TO S4

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT S1;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
mysql> SAVEPOINT S2;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT S3;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+---------------+---------+
| Acc_ID | Holder_Name   | Balance |
+--------+---------------+---------+
|    101 | Aman          |    6500 |
|    102 | Bhavna Sharma |    6000 |
|    104 | Diya          |    4000 |
|    105 | Eshan         |    3000 |
+--------+---------------+---------+
4 rows in set (0.00 sec)

mysql>
mysql> ROLLBACK TO S2;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    6500 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4000 |
+--------+-------------+---------+
4 rows in set (0.00 sec)

mysql>
mysql> INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500);
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> SAVEPOINT S4;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

```
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    7000 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4500 |
|    106 | Farah       |    4500 |
+--------+-------------+---------+
5 rows in set (0.00 sec)

mysql>
mysql> ROLLBACK TO S4;
Query OK, 0 rows affected (0.00 sec)

mysql>
mysql> SELECT * FROM ACCOUNT_TXN;
+--------+-------------+---------+
| Acc_ID | Holder_Name | Balance |
+--------+-------------+---------+
|    101 | Aman        |    6500 |
|    102 | Bhavna      |    6000 |
|    103 | Chirag      |    6000 |
|    104 | Diya        |    4000 |
|    106 | Farah       |    4500 |
+--------+-------------+---------+
5 rows in set (0.00 sec)

mysql>
mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

Additional Challenge (Conceptual)

1. Difference between ROLLBACK and ROLLBACK TO SAVEPOINT

- **ROLLBACK**
  → Cancels the entire transaction
  → All changes are undone
- **ROLLBACK TO SAVEPOINT**
  → Cancels changes only up to that savepoint
  → Earlier changes remain safe

2. Can we rollback to S3 after ROLLBACK TO S2? Why/Why not?

No

Because:

- When you execute ROLLBACK TO S2
- All savepoints created after S2 (like S3) are automatically deleted

3. What happens if COMMIT is executed before creating any savepoint?

All changes become permanent
The transaction ends
Any savepoint created later will not work properly

Lab Sheet: 5

(Exercise on retrieving records from the table)

EMPLOYEES (Employee_Id, First_Name, Last_Name, Email, Phone_Number, Hire_Date, Job_Id, Salary, Commission_Act, Manager_Id, Department_Id)

```
mysql>  CREATE DATABASE company_db;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE company_db;
Database changed
mysql>
mysql> CREATE TABLE EMPLOYEES (
    ->     Employee_Id INT,
    ->     First_Name VARCHAR(50),
    ->     Last_Name VARCHAR(50),
    ->     Email VARCHAR(100),
    ->     Phone_Number VARCHAR(20),
    ->     Hire_Date DATE,
    ->     Job_Id VARCHAR(20),
    ->     Salary INT,
    ->     Commission_Act INT,
    ->     Manager_Id INT,
    ->     Department_Id INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

(a) Find out the employee id, names, salaries of all the employees

```
mysql> SELECT Employee_Id, First_Name, Last_Name, Salary
    -> FROM EMPLOYEES;
+-------------+------------+-----------+--------+
| Employee_Id | First_Name | Last_Name | Salary |
+-------------+------------+-----------+--------+
|         101 | Aman       | Sharma    |   5000 |
|         102 | Bhavna     | Austin    |   4800 |
|         103 | Chirag     | Mehta     |   6000 |
|         104 | Diya       | Austin    |   4500 |
|         105 | Eshan      | Gupta     |   7000 |
|         106 | Farah      | Khan      |   5500 |
+-------------+------------+-----------+--------+
6 rows in set (0.00 sec)
```

(b) List out the employees who works under manager 100

```
mysql> SELECT *
    -> FROM EMPLOYEES
    -> WHERE Manager_Id = 100;
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
| Employee_Id | First_Name | Last_Name | Email           | Phone_Number | Hire_Date  | Job_Id | Salary | Commission_Act | Manager_Id | Department_Id |
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
|         101 | Aman       | Sharma   | aman@gmail.com  | 9876543210  | 2020-01-10 | IT_PROG |  5000 |          NULL |        100 |            60 |
|         102 | Bhavna     | Austin   | bhavna@gmail.com | 9876543211 | 2019-03-15 | HR_REP  |  4800 |          NULL |        100 |            70 |
|         105 | Eshan      | Gupta    | eshan@gmail.com | 9876543214  | 2022-02-18 | SA_REP  |  7000 |          NULL |        100 |            80 |
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
3 rows in set (0.00 sec)
```

(c) Find the names of the employees who have a salary greater than or equal to 4800

```
mysql> SELECT First_Name, Last_Name
    -> FROM EMPLOYEES
    -> WHERE Salary >= 4800;
+-------------+------------+
| First_Name | Last_Name |
+-------------+------------+
| Aman        | Sharma     |
| Bhavna      | Austin     |
| Chirag      | Mehta      |
| Eshan       | Gupta      |
| Farah       | Khan       |
+-------------+------------+
5 rows in set (0.00 sec)
```

(d) List out the employees whose last name is 'AUSTIN'

```
mysql> SELECT *
    -> FROM EMPLOYEES
    -> WHERE Last_Name = 'AUSTIN';
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
| Employee_Id | First_Name | Last_Name | Email           | Phone_Number | Hire_Date  | Job_Id | Salary | Commission_Act | Manager_Id | Department_Id |
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
|         102 | Bhavna     | Austin   | bhavna@gmail.com | 9876543211  | 2019-03-15 | HR_REP  |  4800 |          NULL |        100 |            70 |
|         104 | Diya       | Austin   | diya@gmail.com  | 9876543213  | 2018-07-25 | IT_PROG |  4500 |          NULL |        102 |            60 |
+-------------+------------+----------+-----------------+-------------+------------+--------+--------+---------------+------------+---------------+
2 rows in set (0.00 sec)
```

(e) Find the names of the employees who works in departments 60,70 and 80

```
mysql> SELECT First_Name, Last_Name
    -> FROM EMPLOYEES
    -> WHERE Department_Id IN (60, 70, 80);
+------------+-----------+
| First_Name | Last_Name |
+------------+-----------+
| Aman       | Sharma    |
| Bhavna     | Austin    |
| Chirag     | Mehta     |
| Diya       | Austin    |
| Eshan      | Gupta     |
| Farah      | Khan      |
+------------+-----------+
6 rows in set (0.00 sec)
```

(f) Display the unique Manager_Id.

```
mysql> SELECT DISTINCT Manager_Id
    -> FROM EMPLOYEES;
+-------------+
| Manager_Id |
+-------------+
|         100 |
|         101 |
|         102 |
|         103 |
+-------------+
4 rows in set (0.01 sec)
```

Lab Sheet: 6

(Exercise on updating records in table)

Create Client_master with the following fields (ClientNO, Name, Address, City, State, bal_due)

```
mysql> CREATE DATABASE client_db;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE client_db;
Database changed
mysql>
mysql> CREATE TABLE Client_master (
    ->     ClientNO VARCHAR(10),
    ->     Name VARCHAR(50),
    ->     Address VARCHAR(100),
    ->     City VARCHAR(50),
    ->     State VARCHAR(50),
    ->     bal_due INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

(a) Insert five records

```
mysql> INSERT INTO Client_master VALUES
    -> ('C101', 'Aman', 'Street 1', 'Delhi', 'Delhi', 4000),
    -> ('C102', 'Bhavna', 'Street 2', 'Mumbai', 'Maharashtra', 6000),
    -> ('C123', 'Chirag', 'Street 3', 'Jaipur', 'Rajasthan', 4500),
    -> ('C104', 'Diya', 'Street 4', 'Chennai', 'Tamil Nadu', 7000),
    -> ('C105', 'Eshan', 'Street 5', 'Kolkata', 'West Bengal', 3000);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

(b) Find the names of clients whose bal_due> 5000 .

```
mysql> SELECT Name
    -> FROM Client_master
    -> WHERE bal_due > 5000;
+--------+
| Name   |
+--------+
| Bhavna |
| Diya   |
+--------+
2 rows in set (0.00 sec)
```

(c) Change the bal_due of ClientNO " C123" to Rs. 5100

```
mysql> UPDATE Client_master
    -> SET bal_due = 5100
    -> WHERE ClientNO = 'C123';
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

(d) Change the name of Client_master to Client12.

```
mysql> RENAME TABLE Client_master TO Client12;
Query OK, 0 rows affected (0.01 sec)
```

(e) Display the bal_due heading as "BALANCE"

```
mysql> SELECT ClientNO, Name, Address, City, State, bal_due AS BALANCE
    -> FROM Client12;
+----------+--------+----------+---------+-------------+---------+
| ClientNO | Name   | Address  | City    | State       | BALANCE |
+----------+--------+----------+---------+-------------+---------+
| C101     | Aman   | Street 1 | Delhi   | Delhi       |    4000 |
| C102     | Bhavna | Street 2 | Mumbai  | Maharashtra |    6000 |
| C123     | Chirag | Street 3 | Jaipur  | Rajasthan   |    5100 |
| C104     | Diya   | Street 4 | Chennai | Tamil Nadu  |    7000 |
| C105     | Eshan  | Street 5 | Kolkata | West Bengal |    3000 |
+----------+--------+----------+---------+-------------+---------+
5 rows in set (0.01 sec)
```

Lab Sheet: 7

Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

```
mysql> CREATE DATABASE teacher_db;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE teacher_db;
Database changed
mysql>
mysql> CREATE TABLE Teacher (
    ->     Name VARCHAR(50),
    ->     DeptNo INT,
    ->     Date_of_Joining DATE,
    ->     DeptName VARCHAR(50),
    ->     Location VARCHAR(50),
    ->     Salary INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

(a) Insert five records

```
mysql> INSERT INTO Teacher VALUES
    -> ('Aman', 1, '2020-01-10', 'Mathematics', 'Delhi', 40000),
    -> ('Bhavna', 2, '2019-03-15', 'Commerce', 'Mumbai', 35000),
    -> ('Chirag', 1, '2021-06-20', 'Mathematics', 'Jaipur', 30000),
    -> ('Diya', 3, '2018-07-25', 'Science', 'Chennai', 45000),
    -> ('Eshan', 2, '2022-02-18', 'Commerce', 'Kolkata', 32000);
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

(b) Give Increment of 25% salary for Mathematics Department .

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> UPDATE Teacher
    -> SET Salary = Salary + Salary * 0.25
    -> WHERE DeptName = 'Mathematics';
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

(c) Perform Rollback command

```
mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

(d) Give Increment of 15% salary for Commerce Department

```
mysql> UPDATE Teacher
    -> SET Salary = Salary + Salary * 0.15
    -> WHERE DeptName = 'Commerce';
Query OK, 2 rows affected (0.00 sec)
Rows matched: 2  Changed: 2  Warnings: 0
```

(e) Perform commit command

```
mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)
```

Lab Sheet: 8

(Exercise on order by and group by clauses) Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

```
mysql> CREATE DATABASE sales_db;
Query OK, 1 row affected (0.00 sec)

mysql>
mysql> USE sales_db;
Database changed
mysql>
mysql> CREATE TABLE Sales (
    ->     SalesNo INT,
    ->     Salesname VARCHAR(50),
    ->     Branch VARCHAR(50),
    ->     Salesamount INT,
    ->     DOB DATE
    -> );
Query OK, 0 rows affected (0.02 sec)
```

(a) Insert five records

```
mysql> INSERT INTO Sales VALUES
    -> (1, 'Aman', 'Delhi', 5000, '2000-12-21'),
    -> (2, 'Bhavna', 'Mumbai', 7000, '1999-05-10'),
    -> (3, 'Chirag', 'Delhi', 6000, '2001-12-15'),
    -> (4, 'Diya', 'Chennai', 8000, '1998-08-25'),
    -> (5, 'Eshan', 'Mumbai', 5500, '2000-12-05');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

(b) Calculate total salesamount in each branch

```
mysql> SELECT Branch, SUM(Salesamount) AS Total_Sales
    -> FROM Sales
    -> GROUP BY Branch;
+---------+-------------+
| Branch  | Total_Sales |
+---------+-------------+
| Delhi   |       11000 |
| Mumbai  |       12500 |
| Chennai |        8000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

(c) Calculate average salesamount in each branch

```
mysql> SELECT Branch, AVG(Salesamount) AS Avg_Sales
    -> FROM Sales
    -> GROUP BY Branch;
+---------+-----------+
| Branch  | Avg_Sales |
+---------+-----------+
| Delhi   | 5500.0000 |
| Mumbai  | 6250.0000 |
| Chennai | 8000.0000 |
+---------+-----------+
3 rows in set (0.00 sec)
```

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09

```
mysql> SELECT Salesname, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
    -> FROM Sales
    -> WHERE MONTH(DOB) = 12;
+-----------+-----------+
| Salesname | DOB       |
+-----------+-----------+
| Aman      | 21-Dec-00 |
| Chirag    | 15-Dec-01 |
| Eshan     | 05-Dec-00 |
+-----------+-----------+
3 rows in set (0.00 sec)
```

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
mysql> SELECT Salesname, DOB
    -> FROM Sales
    -> ORDER BY MONTH(DOB);
+-----------+------------+
| Salesname | DOB        |
+-----------+------------+
| Bhavna    | 1999-05-10 |
| Diya      | 1998-08-25 |
| Aman      | 2000-12-21 |
| Chirag    | 2001-12-15 |
| Eshan     | 2000-12-05 |
+-----------+------------+
5 rows in set (0.00 sec)
```

Lab Sheet: 9

Consider the following tables, namely "DEPARTMENTS" and "EMPLOYEES". Their schemas are as follows.

Departments (dept_no, dept_name, dept_location); Employees (emp_id, emp_name, emp_salary, dept_no).

```
mysql> CREATE DATABASE company_db2;
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> USE company_db2;
Database changed
mysql>
mysql> CREATE TABLE Departments (
    ->     dept_no INT,
    ->     dept_name VARCHAR(50),
    ->     dept_location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> CREATE TABLE Employees (
    ->     emp_id INT,
    ->     emp_name VARCHAR(50),
    ->     emp_salary INT,
    ->     dept_no INT
    -> );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> INSERT INTO Departments VALUES
    -> (1, 'HR', 'Delhi'),
    -> (2, 'IT', 'Mumbai'),
    -> (3, 'Sales', 'Chennai');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Employees VALUES
    -> (101, 'Aman', 5000, 1),
    -> (102, 'Bhavna', 7000, 2),
    -> (103, 'Chirag', 6000, 3);
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

a) Develop a query to grant all privileges on the employees table to the departments table.

```
mysql> GRANT ALL PRIVILEGES ON Employees TO 'user1'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

b) Develop a query to grant some privileges on the employees table to the departments table.

```
mysql> GRANT SELECT, INSERT ON Employees TO 'user1'@'localhost';
Query OK, 0 rows affected (0.00 sec)
```

c) Develop a query to revoke all privileges on the employees table from the departments table.

```
mysql> REVOKE ALL PRIVILEGES ON Employees FROM 'user1'@'localhost';
Query OK, 0 rows affected (0.00 sec)
```

d) Develop a query to revoke some privileges on the employees table from the departments table.

```
mysql> REVOKE INSERT, UPDATE
    -> ON Employees
    -> FROM 'user1'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

e) Write a query to implement a savepoint. point.

```
mysql> SAVEPOINT S1;
Query OK, 0 rows affected (0.00 sec)
```

Lab Sheet: 10

Using the tables "DEPARTMENTS" and "EMPLOYEES" perform the following queries

```
mysql>  CREATE DATABASE company_db3;
Query OK, 1 row affected (0.02 sec)

mysql>
mysql> USE company_db3;
Database changed
mysql>
mysql> CREATE TABLE Departments (
    ->     dept_no INT,
    ->     dept_name VARCHAR(50),
    ->     dept_location VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE Employees (
    ->     emp_id INT,
    ->     emp_name VARCHAR(50),
    ->     emp_salary INT,
    ->     dept_no INT
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> INSERT INTO Departments VALUES
    -> (1, 'HR', 'Delhi'),
    -> (2, 'IT', 'Mumbai'),
    -> (3, 'Sales', 'Chennai');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO Employees VALUES
    -> (101, 'Aman', 5000, 1),
    -> (102, 'Bhavna', 7000, 2),
    -> (103, 'Chirag', 6000, 3),
    -> (104, 'Diya', 8000, 2);
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

a) Display the employee details, departments that the departments are same in both the emp and dept.

```
mysql> SELECT *
    -> FROM Employees e
    -> INNER JOIN Departments d
    -> ON e.dept_no = d.dept_no;
+--------+----------+------------+---------+---------+-----------+---------------+
| emp_id | emp_name | emp_salary | dept_no | dept_no | dept_name | dept_location |
+--------+----------+------------+---------+---------+-----------+---------------+
|    101 | Aman     |       5000 |       1 |       1 | HR        | Delhi         |
|    102 | Bhavna   |       7000 |       2 |       2 | IT        | Mumbai        |
|    103 | Chirag   |       6000 |       3 |       3 | Sales     | Chennai       |
|    104 | Diya     |       8000 |       2 |       2 | IT        | Mumbai        |
+--------+----------+------------+---------+---------+-----------+---------------+
4 rows in set (0.00 sec)
```

b) Display the employee name and Department name by implementing a left outer join.

```
mysql> SELECT e.emp_name, d.dept_name
    -> FROM Employees e
    -> LEFT JOIN Departments d
    -> ON e.dept_no = d.dept_no;
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Aman     | HR        |
| Bhavna   | IT        |
| Chirag   | Sales     |
| Diya     | IT        |
+----------+-----------+
4 rows in set (0.01 sec)
```

c) Display the employee name and Department name by implementing a right outer join.

```
mysql> SELECT e.emp_name, d.dept_name
    -> FROM Employees e
    -> RIGHT JOIN Departments d
    -> ON e.dept_no = d.dept_no;
+----------+-----------+
| emp_name | dept_name |
+----------+-----------+
| Aman     | HR        |
| Diya     | IT        |
| Bhavna   | IT        |
| Chirag   | Sales     |
+----------+-----------+
4 rows in set (0.00 sec)
```

d) Display the details of those who draw the salary greater than the average salary

```
mysql> SELECT *
    -> FROM Employees
    -> WHERE emp_salary > (SELECT AVG(emp_salary) FROM Employees);
+--------+----------+------------+---------+
| emp_id | emp_name | emp_salary | dept_no |
+--------+----------+------------+---------+
|    102 | Bhavna   |       7000 |       2 |
|    104 | Diya     |       8000 |       2 |
+--------+----------+------------+---------+
2 rows in set (0.01 sec)
```